

Ex 15 Experiment to understand the data preprocessing in Data science

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Create dataset
data = {
    'Emp_ID': ['E101', 'E102', 'E103'],
    'Name': ['Arun', 'Manasa', 'Naveen'],
    'Age': [25, np.nan, 31],
    'Gender': ['Male', 'Female', 'Male'],
    'Department': ['IT', 'Finance', 'IT'],
    'Salary': ['35,000', '48,000', np.nan],
    'Experience': [2, 6, 7],
    'Performance': [78, 91, 88],
    'City': ['Chennai', 'Coimbatore', 'Chennai']
}

df = pd.DataFrame(data)

# 0. Raw dataset
print("--- 0. RAW DATASET ---")
print(df)

# 1. Data profiling
print("\n--- 1. DATA PROFILING SUMMARY ---")
print("Null Values Count:")
print(df.isnull().sum())

print("\nGender Casing Distribution:")
print(df['Gender'].value_counts())

# Convert Salary to numeric
df['Salary'] = df['Salary'].str.replace(',', '').astype(float)

# Correct Gender casing
df['Gender'] = df['Gender'].replace({
    'male': 'Male',
    'female': 'Female'
})
```

```

})

# Replace invalid performance with NaN
df.loc[df['Performance'] > 100, 'Performance'] = np.nan

# Salary outlier detection using IQR
Q1 = df['Salary'].quantile(0.25)
Q3 = df['Salary'].quantile(0.75)

IQR = Q3 - Q1
upper_limit = Q3 + (1.5 * IQR)

# Cap salary outliers
df['Salary'] = df['Salary'].clip(upper=upper_limit)

# Fill missing Age with median
df['Age'] = df['Age'].fillna(df['Age'].median())

# Fill missing Salary using department median
df['Salary'] = df.groupby('Department')['Salary'].transform(
    lambda x: x.fillna(x.median())
)

# Fill missing Performance with mean
df['Performance'] = df['Performance'].fillna(
    df['Performance'].mean()
)

# Feature Engineering
df['Salary_per_Exp'] = df['Salary'] / df['Experience']

# Create Age Group
df['Age_Group'] = pd.cut(
    df['Age'],
    bins=[0, 28, 100],
    labels=['Young', 'Middle-Aged']
)

# Map cities to regions
region_map = {

```

```

        'Chennai': 'North TN',
        'Coimbatore': 'West TN',
        'Madurai': 'South TN'
    }

df['Region'] = df['City'].map(region_map)

# Smoothing function
def smooth_boundary(s):
    return s.apply(
        lambda x: s.min()
        if abs(x - s.min()) <= abs(x - s.max())
        else s.max()
    )

# Experience smoothing
experience_groups = pd.qcut(
    df['Experience'],
    q=2,
    labels=False,
    duplicates='drop'
)

df['Smoothed_Exp'] = df.groupby(
    experience_groups,
    group_keys=False
)['Experience'].transform(smooth_boundary)

# Min-Max Normalization
minmax = MinMaxScaler()

df['Norm_Performance'] = minmax.fit_transform(
    df[['Performance']]
).ravel()

# Standard Scaling
scaler = StandardScaler()

df['Scaled_Salary'] = scaler.fit_transform(
    df[['Salary']]
)

```

```

).ravel()

# Decimal Scaling
max_val = df['Salary'].abs().max()
j = int(np.ceil(np.log10(max_val)))

df['Decimal_Scaled_Salary'] = df['Salary'] / (10 ** j)

# Data Reduction
df_reduced = df.drop(
    columns=['Emp_ID', 'Name']
)

# City reference table
city_col_table = pd.DataFrame({
    'City': ['Chennai', 'Coimbatore', 'Madurai'],
    'COL_Index': [1.2, 1.0, 0.9]
})

# Data Integration
df_enriched = pd.merge(
    df_reduced,
    city_col_table,
    on='City',
    how='left'
)

# Data Aggregation
df_aggregated = df_enriched.groupby(
    'Department'
).agg(
    Avg_Salary=('Salary', 'mean'),
    Total_Exp=('Experience', 'sum'),
    Avg_Performance=('Performance', 'mean')
).reset_index()

# Validation checks
assert df_enriched.isnull().sum().sum() == 0, \
    "Validation Error: Unhandled NaNs exist!"

```

```

assert (
    (df_enriched['Performance'] >= 0) &
    (df_enriched['Performance'] <= 100)
).all(), "Invalid Performance Score!"

assert (
    df_enriched['Experience'] < df_enriched['Age']
).all(), "Experience exceeds Age!"

assert pd.api.types.is_numeric_dtype(
    df_enriched['Salary']
), "Salary is not numeric!"

# Final output
print("\n--- FINAL ENRICHED ROW-LEVEL DATASET ---")

print(
    df_enriched[
        [
            'Department',
            'Age_Group',
            'Region',
            'Salary_per_Exp',
            'Scaled_Salary',
            'Norm_Performance',
            'COL_Index'
        ]
    ]
)

# Aggregation output
print("\n--- DATA AGGREGATION OUTPUT (DEPARTMENT-LEVEL) ---")
print(df_aggregated)

print("\nALL VALIDATION CHECKS PASSED SUCCESSFULLY!")

```

```

... --- 0. RAW DATASET ---
    Emp_ID   Name   Age   Gender   Department   Salary   Experience   Performance \
0    E101    Arun   25.0   Male     IT           35,000           2           78
1    E102    Manasa  NaN   Female   Finance      48,000           6           91
2    E103    Naveen  31.0   Male     IT           NaN           7           88

    City
0    Chennai
1    Coimbatore
2    Chennai

--- 1. DATA PROFILING SUMMARY ---
Null Values Count:
Emp_ID      0
Name        0
Age         1
Gender      0
Department  0
Salary      1
Experience   0
Performance 0
City        0
dtype: int64

Gender Casing Distribution:
Gender
Male      2
Female    1
Name: count, dtype: int64

```

```

... --- FINAL ENRICHED ROW-LEVEL DATASET ---
    Department   Age_Group   Region   Salary_per_Exp   Scaled_Salary \
0            IT        Young   North TN        17500.0        -0.707107
1        Finance        Young   West TN         8000.0         1.414214
2            IT   Middle-Aged   North TN         5000.0        -0.707107

    Norm_Performance   COL_Index
0            0.000000         1.2
1            1.000000         1.0
2            0.769231         1.2

--- DATA AGGREGATION OUTPUT (DEPARTMENT-LEVEL) ---
    Department   Avg_Salary   Total_Exp   Avg_Performance
0        Finance      48000.0           6           91.0
1            IT      35000.0           9           83.0

ALL VALIDATION CHECKS PASSED SUCCESSFULLY!

```